gRPC, HTTP/2 I HTTP/3

Знизу вгору по стеку: мультиплексування → контракт → дедлайни → статуси → QUIC

Заняття №21

Node.js Pro

Лектор
Кирило Орлов

ДЕВ'ЯТЬ КРОКІВ ЗА ДЕВ'ЯНОСТО ХВИЛИН

Перша половина — транспорт і контракт	Друга половина — що gRPC додав від себе
крок 1 · HTTP/2 голими руками: стріми і HPACK	крок 4 · три типи RPC на одному стрімі
крок 2 · HTTP/1.1 контраст: 6 з ′єднань проти 1	крок 5 · дедлайни: бюджет, який їде на дріт
межі HTTP/2: що не полагодили і що померло	кроки 6-7 · модель помилок і NestJS
крок 3 · контракт: типи, байти, сумісність	крок 8 · HTTP/3 наживо: версія=3 поверх UDP

НАСКРІЗНА НИТКА

Якщо факту не видно на дроті — його не існує. Тому заголовки, трейлери й байти ми сьогодні читаємо очима, а не переказуємо. Кожен крок — окремий процес на портах 22110–22151; усі виводи в деці — з реальних запусків code/21_Grpc.

ПЛАН ЗАНЯТТЯ



- Теза: gRPC не винайшов нічого з того, за що його хвалять
- HTTP/2 голими руками: стріми, непарні id, HPACK
- HTTP/1.1: шість з ґєднань і дві хвилі
- Межі HTTP/2: HoL на TCP, push, пріоритети
- Контракт: wire format, presence, сумісність
- Три типи RPC · дедлайни · модель помилок
- NestJS на gRPC · HTTP/3 наживо · тест

ШІСТЬ ДОКАЗІВ, ЯКІ СЬОГОДНІ НА ЕКРАНІ

01. id 1, 3, 5, 7

чотири запити в одну сесію: три швидкі відповіли ДО повільного

02. 174 Б ×5

таблиця HPACK виросла на першому запиті і далі не росла жодного разу

03. 6 проти 1

HTTP/1.1 купує паралельність з ′єднаннями: 6 і дві хвилі проти 1 і однієї

04. тиха втрата

дві несумісні зміни схеми: одна кидає виняток, друга мовчки псує дані

05. Infinity

дедлайн не поїхав далі — і сервіс B доробив свої 800 мс у нікуди

06. HTTP 200

провальний виклик має статус 200; справжній результат — у трейлерах

ТЕЗА: gRPC НЕ ВИНАЙШОВ НІЧОГО З ЦЬОГО СПИСКУ

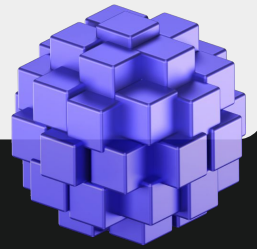
«gRPC швидкий, бо бінарний» — з цього не випливає жодне рішення

мультиплексування — це HTTP/2, і gRPC до нього непричетний

компактність — це protobuf: назв полів на дроті немає взагалі

дедлайн — це звичайний заголовок, а статус — це трейлер

Межі: HTTP/1.1 і TLS — L#3 · REST — L#9 · WebSocket і SSE — L#18 · retry й ідемпотентність — L#22 · mTLS — L#27. Усі стенди сьогодні insecure.



```
ЩО ЧИЙ ШАР?

хвалять gRPC за:
─────────────────
мультиплексування
  → HTTP/2
компактність
  → protobuf
дедлайни
  → заголовок
коди помилок
  → трейлери

─────────────────
gRPC — домовленість,
як цим користуватись
однаково. Дебажити
треба ШАР, не бренд
```

КРОК 1 · HTTP/2 ГОЛИМИ РУКАМИ: СТРІМИ І HPACK

```
// src/01-http2-multiplexing.ts — сервер: голий node:http2
const client = http2.connect('http://127.0.0.1:22110');
await Promise.all([fire('/slow'), fire('/a'),
                   fire('/b'), fire('/c')]);  // /slow — ПЕРШИМ

  порядок ВІДПРАВКИ:  /slow → /a → /b → /c
  порядок ВІДПОВІДЕЙ: /a id=3 · /b id=5 · /c id=7 · /slow id=1
  √ усі три швидкі відповіли ДО повільного — черги немає
  √ id стрімів: 1, 3, 5, 7 — усі НЕПАРНІ (їх відкрив клієнт)

  deflateDynamicTableSize після запитів 1..5:
      174 · 174 · 174 · 174 · 174 байт
  √ таблиця виросла на ПЕРШОМУ запиті і далі не росла
  √ сервер побачив TCP-з'єднань: 1   (на дев'ять запитів)

// лічильник з'єднань — не оцінка, а подія сервера:
server.on('connection', () => { tcpConnections += 1; });
// h2c = HTTP/2 відкритим текстом. Браузерам потрібен TLS з
// ALPN «h2» — але gRPC між сервісами ходить саме так суцільно.
```

сервер — голий node:http2, жодного gRPC: дивимось, що gRPC отримав даром

1 · чотири запити в ОДНУ сесію: /slow спить 300 мс і стартує першим

2 · три швидкі відповіли ПЕРШИМИ — стріми не чекають одне одного

3 · id стрімів 1, 3, 5, 7 — непарні у клієнта, парні в сервера (RFC 9113)

4 · HPACK: таблиця виросла на першому запиті — 174 байти — і далі не росла

5 · стан стиснення спільний на З ′ЄДНАННЯ — звідси CONTINUATION-flood, VU#421644

ОДИН КОРИДОР ПРОТИ ШЕСТИ КОРИДОРІВ

ТРАНСПОРТ

- HTTP/1.1, один сокет
- HTTP/1.1 «як у браузера», 6 сокетів
- HTTP/2 — одна сесія, багато стрімів
- стрім — не окреме з’єднання
- HTTP/3 (QUIC) — доріжки незалежні

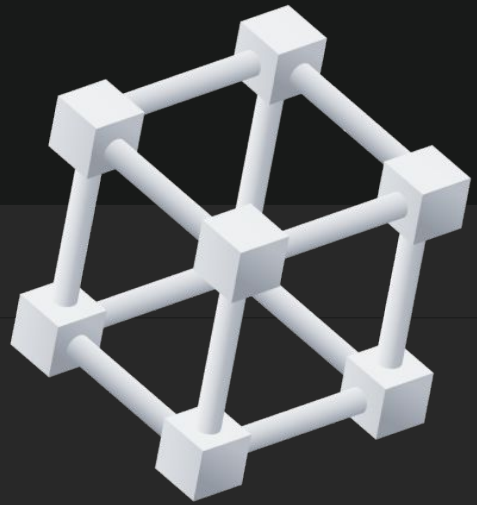
АНАЛОГІЯ (КОРИДОР)

- йдуть гуськом: другий чекає першого
- шість коридорів = шість сокетів
- один широкий коридор, усі одночасно
- номер людини, а не окремий коридор
- цегла падає лише на свою доріжку

Де аналогія ламається: коридор один — і якщо в ньому впала цегла (а цегла тут це втрачений TCP-сегмент), стоять УСІ, навіть ті, кого та цегла не стосується. HTTP/2 прибрав чергу на рівні застосунку і не прибрав її на рівні TCP. Це вміє тільки QUIC — і це крок вісім.



КРОК 2 · HTTP/1.1: ШІСТЬ З′ЄДНАНЬ І ДВІ ХВИЛІ



2.1 · один сокет: усі чотири відповіді ≥ 300 мс — швидкі чекали в черзі

пайплайнінг мав це лікувати: проксі ламали, браузери вимкнули

2.2 · «як у браузера», вісім повільних: з ′єднань 6, хвиль 2

2.3 · ті самі вісім по HTTP/2: з ′єднання 1, хвиля 1. HTTP/2 не «швидший» — він прибрав ліміт на паралельність усередині одного з ′єднання. «Шість на origin» — домовленість браузерів, а не специфікація. Тому L4-балансувальник gRPC не тримає: він роздає з ′єднання, а вантаж їде стрімами



```
// src/02-http1-contrast.ts — той самий сервер, три прогони
const agent1 = new http.Agent({ keepAlive: true, maxSockets: 1 });
const agent6 = new http.Agent({ keepAlive: true, maxSockets: 6 });
// «хвиля» = округлення тривалості до кроку повільної відповіді:
const waves = Math.max(...r.map((ms) => Math.round(ms / SLOW_MS)));
// справжній лічильник з'єднань — це подія сервера, не оцінка:
h1.on('connection', () => { h1conn += 1; });

// ────────── прогін demo/a.sh, такт A2 ──────────
  2.1 · HTTP/1.1, ОДНЕ з'єднання (maxSockets=1)
    ✗ три швидкі ЧЕКАЛИ повільного — усе ≥ 300 мс

  2.2 · HTTP/1.1 «як у браузера»: maxSockets=6, вісім повільних
    ✗ TCP-з'єднань відкрито 6 · хвиль 2

  2.3 · HTTP/2, ОДНА сесія, ті самі вісім запитів
    √ TCP-з'єднань відкрито 1 · хвиль 1

// 6 проти 1 — детерміноване, це лічильник події connection.
// 2 проти 1 хвилі — округлення заміру. Мілісекунди читаємо
// з екрана: вголос «удвічі довше», а не конкретне число.
```

ЩО HTTP/2 НЕ ПОЛАГОДИВ І ЩО В НЬОМУ ПОМЕРЛО

HoL на TCP

ядро не віддасть байти наступних стрімів, поки не дочекається ретрансмісії. Лікує тільки QUIC (RFC 9000)

server push

у специфікації є, у браузерах немає. Замінили 103 Early Hints (RFC 8297). У Node дефолт enablePush: true

пріоритети

дерево з RFC 7540 deprecated у RFC 9113 § 5.3.2. Чинна схема — RFC 9218: urgency та incremental

Захист від Rapid Reset тримається не на числі стрімів
CVE-2023-44487 закрили не лімітом maxConcurrentStreams, а maxSessionMemory і лімітами ресетів — streamResetBurst 1000, streamResetRate 33; вони з ' явились ПІСЛЯ CVE. А enableConnectProtocol: false — це RFC 8441, розширений CONNECT, яким WebSocket їздить поверх HTTP/2 (механіка — L#18).

ДЕФОЛТИ node:http2 — ЩО NODE НЕ ОБМЕЖУЄ

Параметр	Значення	Що з цього випливає
headerTableSize	4096	розмір таблиці HPACK на з ʼєднання
enablePush	true	увімкнено — але push у браузерах мертвий
initialWindowSize	65535	керування потоком на рівні стріму
maxFrameSize	16384	межі DATA-фреймів ≠ межі gRPC-кадрів
maxConcurrentStreams	4294967295	Node НЕ обмежує стріми — захист інший

КОНТРАКТ: ЧОТИРИ ЗМІНИ СХЕМИ, ДВІ З НИХ НЕ ПЕРЕЖИЛИСЬ

01. додали поле 3 — ✓ читається: нове поле прийшло порожнім



02. ПЕРЕЙМЕНУВАЛИ поля — ✓ читається: назв на дроті немає



03. змінили ТИП поля 2 — ✗ ТИХА ВТРАТА: qty прийшов порожнім

04. поміняли НОМЕРИ місцями — ✗ ВИНЯТОК: index out of range: 3 + 8 > 7



05. мантра: номер поля — вічний ідентифікатор. Видалив → reserved і НІКОЛИ не переюзуй; назву міняй скільки завгодно. Жодна зі зламаних змін не сказала «схема несумісна» — тому buf lint і buf breaking стоять у CI

WIRE FORMAT: 12 БАЙТІВ ПРОТИ 37 І РОЗБІР ТЕГІВ

```
об'єкт     {"id":"A-1","qty":7,"currency":"UAH"}
protobuf   0a03412d3110071a03554148  (12 байт)
JSON       той самий об'єкт текстом  (37 байт)

  0a 03 41 2d 31   field=1, wire=2 (bytes) · "A-1"
  10 07            field=2, wire=0 (varint) · 7
  1a 03 55 41 48   field=3, wire=2 (bytes) · "UAH"

  ключ поля = (номер << 3) | wire_type
  ⇒ назв полів на дроті НЕМАЄ
  ⇒ перейменувати можна, змінити НОМЕР — ні

── FIELD PRESENCE ──────────────────────────
  порожній дріт   без optional → {"qty":0}
                  з optional   → {}
  явний нуль      обидва       → {"qty":0}
  ⇒ «нуль» ≠ «не передали»? поле МУСИТЬ бути optional
```



```
// src/03-proto-codegen.ts — контракт → типи → байти → CI
// protoc системно НЕ потрібен: компілятор protobuf вбудований
// у buf, а buf і ts-proto — звичайні npm-залежності.
// [buf] src/generated/marketplace/v1/marketplace.ts — 942 рядків

const probe: Order = { id: 'A-1', quantity: 7 };  // друкарська
// ✗ компілятор зупинив збірку:
//   probe.ts(2,36): error TS2353: Object literal may only specify
//   known properties, and 'quantity' does not exist in type 'Order'.

// 3.4 · писар (стара схема) відправив: 0a03412d311007
//   √ додали поле 3        читається
//   √ ПЕРЕЙМЕНУВАЛИ поля   читається
//   ✗ змінили ТИП поля 2   ТИХА ВТРАТА
//   ✗ поміняли НОМЕРИ      ВИНЯТОК: index out of range: 3 + 8 > 7
//   ✗ несумісних сценаріїв 2: тихих втрат 1, винятків 1

// 3.5 · ✗ buf breaking зупинив зміну ЩЕ ДО МЕРЖУ:
//   marketplace/v1/marketplace.proto:38:3: Field "2" with name
//   "qty" on message "Order" changed type from "int32" to "string".
```





```
// src/04-rpc-kinds.ts — сервер тут звичайний node:http2,
// щоб заголовки було видно ОЧИМА, а не через абстракцію.

  4.1 · заголовки запиту на дроті:
    :method                 POST
    :path                   /marketplace.v1.Orders/GetOrder
    content-type            application/grpc
    te                      trailers
    grpc-accept-encoding    identity,deflate,gzip
    grpc-timeout            <залишок часу> ← число + одиниця

  тіло (hex): 000000000a0a086f726465722d3432
    [0]    = 0   ← compressed-flag (1 байт)
    [1..4] = 10  ← довжина повідомлення (4 байти, big-endian)
    [5..]  = 0a086f726465722d3432   ← сам protobuf

// «te: trailers» підписаний у специфікації gRPC дослівно:
// "Used to detect incompatible proxies" — це перевірка на вході.
// Межі DATA-фреймів HTTP/2 з межами gRPC-кадрів НЕ пов'язані.
```

ТРИ ТИПИ RPC: РІЗНИЦЯ — СЛОВО stream У .proto

Тип	Що на дроті	Що на екрані
unary	один стрім, одне повідомлення	одна відповідь — 95% реального gRPC-трафіку
server-streaming	один стрім, потік в один бік	5 подій, кожна окремим повідомленням
client-streaming	один стрім, потік в один бік	у контракті свідомо немає: механізм той самий, що в bidi
bidirectional	два незалежні напрямки в одному стрімі	3 надіслані, 3 підтвердження; перше — МІЖ нашими записами
усі чотири	ОДИН HTTP/2-стрім на виклик	різниця в .proto — слово stream

ДЕДЛАЙНИ: ТРИ СЦЕНАРІЇ І ТРИ ВИСНОВКИ

5.1 · дефолт: [B] МІЙ дедлайн — Infinity. Не 30 секунд — НЕСКІНЧЕННІСТЬ, це в специфікації gRPC

5.2 · пропагації немає: клієнт пішов через 300 мс з DEADLINE_EXCEEDED, а B доробив усі 800 мс у нікуди

5.3 · один рядок лікує: B бачить СКІНЧЕННИЙ залишок і сам себе зупиняє

висновок 1 · автопропагації в Node немає: у Go і Java це context — тут або руками, або інтерсептор, і ніхто вам про це не нагадає

висновок 2 · на дріт їде ЗАЛИШОК часу, а не момент — тому роз ′їхані годинники двох машин нічого не ламають

висновок 3 · DEADLINE_EXCEEDED НЕ означає, що операція не виконалась. Ідемпотентність — властивість ОПЕРАЦІЇ, ключ — ІНСТРУМЕНТ

```
// src/05-deadline-propagation.ts — A (край) → B (склад)
// БІК A: уся «пропагація дедлайну» в Node — це ОДИН рядок
const opts: grpc.CallOptions = {};
if (PROPAGATE) {
  opts.deadline = call.getDeadline();   // ⭐ ось він
}            // grpc-js сам перерахує його в ЗАЛИШОК часу
             // і покладе в заголовок grpc-timeout

// БІК B: що він бачить у СВОЄМУ дедлайні
const d = call.getDeadline();
seen = d === Infinity ? 'Infinity' : 'мс лишилось';
if (call.cancelled) log('B', 'кидаю роботу');

// 5.1  [B] МІЙ дедлайн: Infinity   ✗ дефолт НЕСКІНЧЕННИЙ
// 5.2  [B] МІЙ дедлайн: Infinity   ✗ доробив усі 800 мс
//      клієнт: DEADLINE_EXCEEDED   ✗ call.cancelled=false
// 5.3  [B] МІЙ дедлайн: <залишок>  √ call.cancelled=true
//      √ B побачив КІНЦЕВИЙ дедлайн і сам себе зупинив
// Автопропагації в Node немає — у Go і Java це context.
// Тут або руками, або інтерсептор. Ретрай після дедлайну — L#22
```



КРОК 6 · ПРОВАЛЬНИЙ ВИКЛИК ВІДДАЄ HTTP 200

чому так: на момент відправки заголовків результату ЩЕ немає

тому статус їде ПІСЛЯ тіла, у трейлерах: транспорт спрацював

дашборд «5xx на балансувальнику» для gRPC покаже рівно нуль

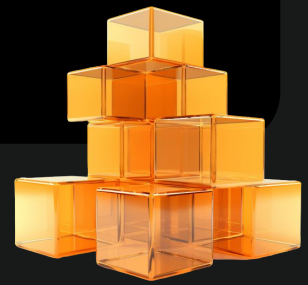
ALB і nginx, що ретраять по 5xx, не побачать ЖОДНОЇ помилки

трейлери вміє не кожен проксі — тому клієнт шле te: trailers

зворотний бік: 502 означає, що виклик НЕ дійшов до застосунку

```
КРОК 6, ТАКТ 6.3 — РЕАЛЬНИЙ ПРОГІН
HTTP :status   → 200
grpc-status    → 7 (PERMISSION_DENIED)
grpc-message   → %D1%87%D1%83%D0%B6%D0%B5…
   decodeURIComponent → чуже замовлення

✗ виклик ПРОВАЛИВСЯ, а HTTP-статус — 200
√ результат лежить у ТРЕЙЛЕРАХ, після тіла
──────────────────────────────────
6.1 · кодів у @grpc/grpc-js: 17
6.2 · gRPC ⇔ REST — домовленість,
      а не стандарт:
  5 NOT_FOUND            404
  8 RESOURCE_EXHAUSTED   429
  9 FAILED_PRECONDITION  409
```



17 КОДІВ: КОД ПОМИЛКИ — ЦЕ ІНСТРУКЦІЯ КЛІЄНТУ

КОД / МЕЖА	ЩО ЦЕ КАЖЕ КЛІЄНТУ
• UNAVAILABLE • ABORTED • FAILED_PRECONDITION • DEADLINE_EXCEEDED • exactly-once доставки	• повторити САМЕ ЦЕЙ виклик • повторювати рівнем вище • НЕ повторювати до виправлення стану • могла виконатись → повтор ідемпотентний • не існує: at-least-once + ідемпотентний ефект

```
// src/07-nestjs/main.ts — гібрид: HTTP-шлюз + gRPC-мікросервіс
app.connectMicroservice<MicroserviceOptions>({
  transport: Transport.GRPC,   // ← три поля, і це весь транспорт
  options: { url: '127.0.0.1:22150', package: 'marketplace.v1',
             protoPath: PROTO_PATH, loader: { longs: String } },
});      // ↑ без longs int64 приїде Long-об'єктом, а не рядком

@GrpcMethod('Orders', 'GetOrder')   // зв'язок — ІМ'Я з .proto,
                                    // а не назва класу
throw new RpcException({ code: GrpcStatus.NOT_FOUND, ... });
// HttpException тут → клієнт отримає код 2 (UNKNOWN):
// HTTP-виняток у gRPC мапити просто нема куди.

//   GET /fail/not_found → grpcCode 5 · denied → 7 · exhausted → 8
//   ✓ grpcCode доїхав у шлюз БЕЗ перекладу — той самий код
//   ✓ Nest не додає до протоколу нічого: під капотом grpc-js

// src/08-http3.ts — крок 8, єдине місце, де потрібен Docker
//   8.1 require('node:quic') → No such built-in module: node:quic
//   8.2 версія=3 код=200   ✓ справжній QUIC поверх UDP
//   8.3 версія=2 код=200   ✓ той самий порт віддає h2 і h3
```

HTTP/3 НАЖИВО: ЧЕСНА МЕЖА, ДОКАЗ І ЩО ЗА ДУЖКАМИ

ЧЕСНА МЕЖА: у Node 22 модуля node:quic немає взагалі, а системний curl зібраний без HTTP/3. У стеку курсу HTTP/3 не підняти нічим — тому готові образи, а не чужий скріншот. gRPC поверх HTTP/3 теж не їздить

ДОКАЗ І МЕЖА: версія=3 код=200, і той самий порт віддає версія=2. Образ клієнта під емуляцією — крок доводить факт, а не швидкість

ЗА ДУЖКАМИ: 0-RTT можна повторити атакою (RFC 9001) · connection migration — за Connection ID, а не за IP:порт · QPACK замість HPACK · Alt-Svc: про HTTP/3 дізнаються по дорозі, без раунд-тріпу — DNS-запис HTTPS



ПІДСУМКИ, ТЕСТ І МІСТ ДО L#22

ТРАНСПОРТ

мультиплексування побачили БЕЗ жодного gRPC: одне з ′єднання, багато стрімів. HoL на TCP лишився

ДОМОВЛЕНОСТІ

дедлайн — заголовок із залишком, статус — трейлер, HTTP-статус провального виклику — 200

ДАЛІ

L#22 — розподілені патерни. DEADLINE_EXCEEDED НЕ означає, що операція не виконалась

ТЕСТ #21

2 бали · 6 питань · одна спроба · прохідний ≥ 5/6 · дедлайн 17.11. Теми: wire format · три типи RPC · мультиплексування HTTP/2 · HTTP/3 · gRPC проти REST · пропагація дедлайну

КОНТРАКТ

назв полів на дроті немає: номер вічний, назва — ні. Сумісність перевіряє машина — buf breaking у CI

Q&A



r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ